

FORENSICS INVESTIGATION WRITEUP

The Byte Lotus Hotel · Management Wants a Word

Hacker Holidays 2026 · Windows Forensics · DPAPI · VeraCrypt · Crypto · Pixel
Forensics

Room	Hacker Holidays 2026 · The Byte Lotus Hotel · Management Wants a Word
Category / Difficulty	Forensics / Hard (120 points)
Evidence	management-wants-a-word-forensics-hh-day-14.zip — KAPE triage of Vera's Windows machine: registry hives (SAM/SYSTEM/SECURITY), Chrome profile, DPAPI master key, and a suspicious 100 MB file called <i>backup</i>
Scenario	Housekeeping found a guest's laptop after an early checkout. Room 214, registered to a "Vera." IT pulled a full triage before wiping it for the next guest. Somewhere in that trail is a password she never meant to leave behind.
Objective	Hunt down the artifacts scattered across Vera's machine and figure out how they fit together. Follow the trail to the password → open the hidden door → claim the flag.
Flag	THM{1t_w4s_V3r4_A11_Al0ng?!}
Author	Sameekey
Date	10 August 2026

Executive summary. This room is a five-layer onion of forgotten secrets. A browser that remembers credentials she "never told anyone"... an LSA *DefaultPassword* she forgot to remove... a DPAPI master key that her own password unlocks... a VeraCrypt volume whose **primary header was deliberately destroyed** (only the hidden backup header survives)... and inside it, a PDF invoice that is just a flat image — with a flag whose final characters had to be dragged out of the pixels by hand. This writeup replays every command with real output, including the dead ends.

1 · The Scene: A KAPE Triage Dump

The challenge ships a single zip. Inside is a standard KAPE-style triage of **C:\Users\vera** plus the machine hives — exactly what a forensic image looks like before a wipe. First move: list everything that is not OS noise.

```
unzip -l management-wants-a-word-forensics-hh-day-14-1785854680266.zip
|-- KAPE/C/Users/vera/Documents/backup 100 MB
|-- KAPE/C/Users/vera/AppData/Roaming/Microsoft/Protect/<SID>/ DPAPI master key
|-- KAPE/C/Users/vera/AppData/Local/Google/Chrome For Testing/User Data/ History, Login Data, Local State
|-- KAPE/C/Windows/System32/config/ SAM SYSTEM SECURITY hives
`-- KAPE/C/Users/vera/NTUSER.DAT + LOGs user hive
```

Key insight #1. The user directory for this machine's Chrome lives under “Chrome For Testing” — a full Chrome install, so all the usual artifacts (History, Login Data, Local State, Sessions) are present. KAPE preserved the profile as-is, including the encrypted password store.

Key insight #2. *Documents/backup* is 100 MB of what file calls “data” and what entropy analysis calls “random”:

```
file KAPE/C/Users/vera/Documents/backup
data
xxd -l 32 KAPE/C/Users/vera/Documents/backup
00000000: f372 f7cc d607 4b17 a8aa 8865 12af abdf .r....K....e....
00000010: f293 9a74 72ea acbc bee5 b479 4c88 5c7f ...tr.....yL.\
00000020: ...
```

Incompressible bytes = almost certainly an **encrypted container**. Which product? The story hint nails it: *“why did Patch tell me this version number 1.26.29 idk what it means”* — that is **VeraCrypt 1.26.29**. Vera keeps a VeraCrypt volume in her own Documents folder, and the password to it is hiding somewhere in the machine's memory.

2 · Chrome's Memory: Saved Credentials

The story tweet practically names the technique: *“a browser will remember things for you that you never told anyone else... not every hidden file needs a password cracker, some of them just need a really good memory.”* Chrome's **Login Data** SQLite database is that memory.

```
sqlite3 '.../User Data/Default/History' \
"SELECT url, title, visit_count FROM urls ORDER BY last_visit_time DESC LIMIT 5;"

https://www.google.com/search?q=how+to+exfiltrate+data+red+teaming | 4 visits
https://www.google.com/search?q=chrome+cves | 2 visits
http://bytelotus.thm:8080/ | SecureVault Portal | 2 visits
http://bytelotus.thm:8080/login | Error response | 1 visit
```

An internal vault portal at **bytelotus.thm:8080**, and Vera browsing “how to exfiltrate data” and “chrome cves” beforehand. Now the saved credential:

```
sqlite3 '.../User Data/Default/Login Data' \
"SELECT origin_url, username_value, hex(password_value) FROM logins;"

http://bytelotus.thm:8080/ | VeraSecretVault |
763130 C88A72A64F35F63E883EA0A7F6 4A6870E46B0BBB469A756EDA88B7E324
C3E1C51015AA6FD8D65AC48961E1EA32 4CE1707807FEB3D7
```

Key insight #3. The blob begins with hex **76 31 30** = ASCII "**v10**" — Chrome's AES-256-GCM password format. On Windows the GCM key is itself wrapped by **DPAPI**: the 32-byte key lives in *Local State* under *os_crypt.encrypted_key*, protected by a DPAPI blob that only the user's **DPAPI master key** can open. And that master key is encrypted with — wait for it — Vera's Windows password.

3 · The Password She Forgot: LSA DefaultPassword

Vera's Windows password is nowhere in the browser... but Windows itself keeps a copy. When auto-logon is configured, the logon password is written into the LSA **DefaultPassword** secret — recoverable from the SECURITY hive. The LSA secrets are encrypted with a key derived from the system bootkey (SYSTEM hive), so the triage conveniently includes both.

```
impacket.secretsdump -sam SAM -system SYSTEM -security SECURITY LOCAL

[*] SYSTEM hive secrets
Boot Key: 0f6f73ce89c8cda52d06fcc5131e040f
[*] SAM hive secrets
vera:1000:aad3b435b51404eeaad3b435b51404ee:1241186a4aac4f34f4bf7ace71b396a8:::
[*] LSA Secrets
DefaultPassword (Unknown User): minivera <-- there it is
DPAPI_SYSTEM ... NL$KM ...
```

Vera's auto-logon password was **minivera**. Her DPAPI master key is protected by exactly this password (DPAPI derives the user key from SID + password hash), so the whole vault is now one decryption step away.

4 · Cracking Open the DPAPI Master Key

Vera's DPAPI master key file: `%APPDATA%\Microsoft\Protect\S-1-5-21-2529683458-431225740-1723070931-1000\c90719ef-5b98-474e-b934-136d606a702a`. Header parse reveals modern protection: PBKDF2 with **8000 rounds**, **CALG_SHA_512** (0x800e) PRF and **CALG_AES_256** (0x6610) — not the legacy 3DES/SHA1 most writeups assume. But the password is the password:

```
python3 - <<'PY'
from dpapick3 import masterkey
mkf = masterkey.MasterKeyFile(open('c90719ef-5b98-474e-b934-136d606a702a', 'rb').read())
mkf.decryptWithPassword('S-1-5-21-2529683458-431225740-1723070931-1000', 'minivera')
print(mkf.get_key().hex())
PY

5e5715ec9b6df5a86e97902692a66d28e691f05d5bc1e04d0159cfe960e94c9
7c07e5004a0179d3a96df2468885a28175b0b02cc064445f116a752d2b3e9d40

# the block-level crypto, for the curious:
# prekey = HMAC-SHA1( SHA1(pwd UTF-16LE), SID + '\0' )
# key/iv = PBKDF2-HMAC-SHA512(prekey, salt, rounds=8000, dklen=48)
# AES-256-CBC decrypt -> hmacSalt(16) + hmac(64) + masterkey-block
# verify hmac == HMAC-SHA512(HMAC(prekey,hmacSalt), block)
```

The 64-byte decrypted blob contains the real 32-byte master key material — now we can unwrap anything DPAPI protected for this user, including Chrome's vault key.

5 · Chrome's Vault Key → The Password

Chrome stores its AES key in **Local State** as a Base64 string prefixed with five bytes of ASCII **DPAPI**. Strip the prefix, decrypt the blob with the master key:

```
jq -r '.os_crypt.encrypted_key' '.../User Data/Local State' | base64 -d | tail -c +6 >
chrome-key.dpapi
python3 - <<'PY'
from impacket.dpapi import DPAPI_BLOB
blob = DPAPI_BLOB(open('chrome-key.dpapi', 'rb').read())
key = blob.decrypt(bytes.fromhex(MASTERKEY))
print(key.hex())
PY

206a39a0971327ea9487e4aea9844f5d3670162456982276939a712646da0b02 (32-byte AES-256 key)
```

Now the v10 credential from *Login Data*. Layout: "**v10**" + **12-byte nonce** + ciphertext with a **16-byte GCM tag** appended. One subtlety hit during analysis: `decrypt_and_verify(ct, None)` failed with a MAC error because the tag length is implicit; splitting the tag off explicitly works.

```
python3 - <<'PY'
from Crypto.Cipher import AES
blob = bytes.fromhex(HEX_PASSWORD_VALUE)
nonce, ct = blob[3:15], blob[15:]
key = bytes.fromhex(AES_KEY)
print(AES.new(key, AES.MODE_GCM, nonce).decrypt_and_verify(ct[:-16], ct[-16:]))
PY

b'Wh4t1sV3raD0inG0nTh1sH0st'
```

Vault password recovered: Wh4t1sV3raD0inG0nTh1sH0st — “What is Vera doing on this host?” — the password she never meant to leave behind. Time to open the door.

6 · VeraCrypt: Three Strikes, Then a Backup Header

Linux's `cryptsetup` opens VeraCrypt volumes natively. Attempt one, primary header:

```
sudo cryptsetup tcryptOpen --veracrypt -d /tmp/vkey \
'KAPE/C/Users/vera/Documents/backup' vera_backup

Enter passphrase for .../backup:
No device header detected with this passphrase. <-- strike one
```

The passphrase is definitely right (it decrypted a GCM-authenticated blob — a wrong key could not have verified). So the volume itself is special. Two suspects: a **hidden volume** (Vera was “keeping something very quiet”...) or a destroyed primary header. Try hidden:

```
sudo cryptsetup tcryptOpen --veracrypt --tcrypt-hidden '.../backup' vera_hidden
Enter passphrase for .../backup:
No device header detected with this passphrase. <-- strike two
```

Strike three is not a strike. VeraCrypt keeps an **emergency backup of the volume header** near the end of the container — a legit user can restore a corrupted primary header from it. An attacker who wants her secrets to survive her own “backup” would kill the primary header on purpose, leaving the backup header as the only keyhole.

```
sudo cryptsetup tcryptOpen --veracrypt --tcrypt-backup '.../backup' vera_bb
Enter passphrase for .../backup: <-- SUCCESS
sudo mkdir -p /mnt/vera && sudo mount -o ro /dev/mapper/vera_bb /mnt/vera
ls -la /mnt/vera/
```

```
$RECYCLE.BIN/ secret_financial_documents/ System Volume Information/  
ls -la /mnt/vera/secret_financial_documents/  
important_invoice_byte_lotus.pdf 26747 B  
transactions_q3.csv 427 B
```

A hidden volume was a red herring — the real twist is a **deliberately destroyed primary header**. Vera can always restore the backup header; forensics investigators now can too.

7 · The Secret Financial Documents

The volume holds a small folder named *secret_financial_documents*. The CSV first — something is off with one row:

```
cat transactions_q3.csv  
Date,Reference,Vendor,Description,Amount,Status  
2026-07-02,TXN-10481,Byte Lotus Catering,Staff refreshments,842.16,Approved  
2026-07-05,TXN-10493,Sunrise Transport,Airport transfers,1260.00,Approved  
2026-07-09,TXN-10514,Lotus Printworks,Event materials,418.75,Approved  
2026-07-12,TXN-10531,Internal Adjustment,Image asset correction,0.00,Archived  
2026-07-15,TXN-10547,Byte Lotus Resorts,Guest accommodation,3840.00,Approved
```

A **0.00 / “Archived”** “Internal Adjustment” labelled *Image asset correction* — the classic embezzlement pattern: money moved, then “corrected” to zero and archived. The invoice PDF is presumably the receipt. But pdftotext returns nothing — the PDF is a single flat image, 477×543 pt, no text layer, MuPDF-produced, with one drawn XObject (*Img3*, 636×724 grayscale + alpha + ICC profile). An image-only invoice on purpose.

8 · The Flag Hunt: OCR is Not Enough

Render the page and OCR the embedded image. First extraction:

```
python3 - <<'PY'  
import fitz  
doc = fitz.open('important_invoice_byte_lotus.pdf')  
img = doc.extract_image(3) # xref 3 = Img3  
open('invoice.png', 'wb').write(img['image'])  
PY  
python3 - <<'PY'  
from rapidocr_onnxruntime import RapidOCR  
for line in RapidOCR>('invoice.png')[0]: print(line[1])  
PY  
  
INVOICE ... BILL TO: Hotel Cleaning LLC ... PAYABLE TO: Byte Lotus Resorts ...  
NO. DESCRIPTION TOTAL  
1. Flag: THM[1t_w4s_V3r4_A11_AI0ng?!] $100  
TOTAL AMOUNT: $100
```

The invoice is a single line item: the flag itself, plus the \$100 fake invoice. But look at the brackets and the last word: [...] and **A10ng** vs **A10ng** vs **Ai0ng** — three OCR runs at different scales gave *three different spellings*. The flag format demands {...} and a 2-3-4-3-7 segment split. OCR lies; pixels don't. Time for manual glyph forensics.

The ambiguous glyphs: the brace pair, the second character of **A?0ng** (1 / l / i / ?) and the oval (0 / O?). Each candidate character is present elsewhere in the invoice — extract a reference glyph for each and

compare pixel profiles.

```
# threshold the flag line, split into glyphs by empty columns, print as ASCII art
th = np.array(Image.open('invoice.png').convert('L'))[448:462,:] < 200
...
```

Reference glyphs harvested from the same document (all same font, same size):

Candidate	Reference from the invoice	Verdict
1	..##. .###.. ..##..##.. ##### # (5 px wide, top serif, bottom bar – from “2122”)	NOT it — 5 px wide with serif+bar
l	# ## (1-2 px, full height + ascender, from “Hotel Cleaning LLC”)	MATCH — 1-2 px stroke, ascender
i	## then # (dot ABOVE a short body, from “Cleaning”)	NOT it — has a dot, body starts below x-height
0	..###.. .#...#. #.....# ← from “8/09/2026” #.....# .#...#. ..###..	MATCH — pixel-identical to the flag glyph
O	##### ..##.. (thick sides, from ..##.. “INVOICE”)	NOT it — thick-sided, 6 px

The brace pair: the opening glyph (x≈142-145) shows the classic pinched waist of { — the left curl and right body squeeze to a single dark pixel mid-height (grayscale 145→98→157), which is why aggressive thresholding dropped it to “1 px”. The closing glyph is a double curl with a gap at mid-height: }. So:

```
{ 1t _ w4s _ V3r4 _ A11 _ A10ng ?! }
|_| |_| `---|---v |_| `-----|-----v
2 3 4 3 7 <-- matches the room's format: ***{**_**_****_**_*****}
```

FLAG THM{1t_w4s_V3r4_A11_A10ng?!}

“It was Vera all along?!” — the vault account name (*VeraSecretVault*), the fake invoice for \$100, the 0.00 “archived adjustment” in the ledger, and the sabotaged volume header: Vera has been funnelling money out of Byte Lotus Resort and built an entire forensic escape room to hide it. Management wanted a word; they got a whole conversation.

9 · Full Kill-Chain Summary

Step	What was done	Result
1	List KAPE triage; file-type + entropy-check <i>backup</i>	100 MB encrypted container; hint → VeraCrypt 1.26.29
2	Dump Chrome History + Login Data	Vault portal bytelotus.thm:8080, user VeraSecretVault, v10-encrypted password
3	secretsdump SAM/SYSTEM/SECURITY	LSA DefaultPassword: <i>minivera</i>

4	Decrypt DPAPI master key (AES-256/SHA-512, 8000 rds)	Master key 0x5e5715ec...9d40
5	Decrypt Local State os_crypt DPAPI blob	Chrome AES-256 key 0x206a39a0...b02
6	AES-256-GCM decrypt v10 blob (tag split)	Vault password Wh4t1sV3raD0inG0nTh1sH0st
7	tcryptOpen: primary <i>X</i> , hidden <i>X</i> , --tcrypt-backup ✓	Mounted volume; secret_financial_documents/
8	Extract image-only invoice PDF; OCR + glyph forensics	Flag recovered from pixels (OCR alone was wrong)

10 · Key Takeaways

- **Chrome saved passwords are not “hashed” — they're DPAPI-wrapped.** The v10 blob is AES-256-GCM under a key that the OS keeps behind DPAPI. Anyone with the user's DPAPI master key (i.e. her Windows password or a memory dump) can unwrap the entire vault. No cracking required.
- **DefaultPassword never lies.** The LSA secret in the SECURITY hive stores the auto-logon password (almost always the account password) in recoverable plaintext. Always dump LSA secrets on a Windows triage.
- **Modern DPAPI ≠ 3DES.** This master key used AES-256-CBC with a SHA-512 PRF at 8000 PBKDF2 rounds — read the header flags (0x800e / 0x6610) before assuming the classic SHA1+3DES path.
- **Chrome v10 layout:** "v10" + 12-byte nonce + ciphertext || 16-byte tag. When decrypt_and_verify(ct, None) fails, split the tag explicitly — GCM tag handling is implementation-sensitive.
- **VeraCrypt keeps a backup header.** cryptsetup tcryptOpen --veracrypt --tcrypt-backup reads it. If the primary header fails with a known-good passphrase, suspect sabotage and try --tcrypt-backup (and --tcrypt-hidden).
- **OCR is a suggestion, not evidence.** Three OCR passes gave three spellings of the same flag word. When the flag format constrains character counts, do glyph-level comparison against reference glyphs in the same document — pixels are the ground truth.

© 2026 Sameekey — TryHackMe “Hacker Holidays 2026 · Management Wants a Word” writeup. All findings reproduced from the live challenge artifacts.